running any data or structure modifications. Discuss a rollback plan when that's appropriate.

For queries, consider doing all exploration queries on a replica database. Replicas run in a *read only* mode so destructive DML operations like TRUNCATE aren't possible and result in an error.

Consider creating a read only PostgreSQL user on the primary database using the pg_read_all_data or pg_monitor role. The article *Creating a Read-Only Postgres User*[12] from Crunchy Data describes how to do this. By regularly using a read only user, you'll add another safety mechanism that prevents accidental modifications.

If you're creating database tables that are temporary and should be removed later, create the tables in their own schema. You'll use that technique in this book by creating tables within a temp schema.

A schema named temp helps reveal the temporary nature to other team members, which helps them more easily clean it up later if you forget to!

Almost all of the content in your production PostgreSQL database for your Rails application should have a corresponding Active Record Migration file. Besides the separate temporary schema, tables created in the application schema (public by default) that do not have corresponding migrations can probably be removed.

One technique to assist in removing tables is to rename them first using an ALTER TABLE statement. A renamed database table that doesn't match the table name expected by the application will produce errors. To roll this change back quickly, run an ALTER TABLE again that renames it back to the original name.

For example, from psql connect to your experiments database and rename the tbl you created earlier.

Run this statement:

```
ALTER TABLE tbl RENAME TO tbl_pending_delete;
```

This could be an intermediate step that allows you to quickly rename it back if needed. If no application errors are observed after a period of time, you can confidently drop the table by running DROP TABLE tbl_pending_delete;.

You now have several techniques to add safeguards when working directly with your production database.

12. https://www.crunchydata.com/blog/creating-a-read-only-postgres-user